



PUCP

Ingreso y salida de datos

Sesión de repaso



Objetivos de la sesión de clase

- Repasar el patrón para relacionar dos archivos CSV mediante una llave común, releyendo el archivo secundario con `clear()` y `seekg(0)`.
- Repasar la lectura manual de un archivo JSON, aplicando `ignore()` hasta cada separador y `>>` para extraer los valores.
- Repasar la generación de un archivo JSON a partir de un CSV y su relectura para producir un reporte final.
- Resolver dudas puntuales de código sobre los tres ejercicios antes de la práctica calificada.



Ejercicio 1: Relacionar dos CSV por una llave común

CSV --> TXT

- ❌ Dos archivos comparten una llave: **productos.csv** (id, categoría) y **movimientos.csv** (idProducto, tipo E/S, cantidad), relación uno a muchos.
- ❌ Por cada producto, recorrer movimientos.csv completo (volviendo a leer con `clear() + seekg(0)`) acumulando entradas y salidas.
- ❌ Calcular el stock resultante de cada producto (entradas – salidas); indicar explícitamente si un producto no tiene movimientos.
- ❌ Al final, imprimir el total de entradas, el total de salidas y el producto con mayor stock resultante.



Ejercicio 1: Puntos a repasar

- ✗ La lectura de (**productos.csv**) va en main; la lectura de (**movimientos.csv**) vive dentro de acumularMovimientosDeProducto.
- ✗ `movimientos.clear(); movimientos.seekg(0);` antes de cada relectura completa del archivo secundario.
- ✗ La función de acumulación puede devolver un int con la cantidad de movimientos encontrados, para detectar productos sin movimientos. ✓
- ✗ Solo `ifstream/ofstream`, sin arreglos ni string: cada movimiento se procesa en el momento en que se lee.



Ejercicio 2: Lectura de un archivo JSON

JSON --> TXT

- ✗ **turnos.json** contiene un registro por turno, en formato JSON Lines: {"id": ..., "turno": ..., "pacientes": ..}, con dos turnos, M (mañana) y T (tarde). -
- ✗ Cada campo se ubica igual que en CSV: descartar hasta el separador (dos puntos o comillas) con ignore(), y extraer el valor con >>. -
- ✗ Acumular por separado el turno M y el turno T: cantidad de registros, suma, mínimo y máximo de pacientes atendidos. ✓
- ✗ Generar el reporte alineado, igual que en los ejercicios de CSV de la sesión anterior.



PUCP

Ejercicio 2: Puntos a repasar

- ✗ `archivo.ignore(TAM_LINEA, ':'); archivo >> valor;` es el patrón para cada campo, sin leer la clave como texto.
- ✗ Las claves se descartan siempre con el mismo `ignore()`, nunca se comparan como texto.
- ✗ Los valores numéricos no llevan comillas; los de texto sí, confundir el tratamiento de ambos es el error más común.
- ✗ El ciclo se controla con `archivo.peek()` para saber si el siguiente carácter marca el inicio de otra línea JSON.



Ejercicio 3: de CSV a JSON y de vuelta a un reporte

CSV --> JSON --> TXT

- ✗ A partir de **reservas.csv** (fecha, hora, id, tipo de habitación, noches, precio por noche), calcular el importe de cada reserva.
- ✗ Generar un archivo JSON Lines con un objeto por reserva, validando la fecha y hora antes de aceptarla como válida.
- ✗ El mismo programa vuelve a abrir el JSON generado y lo **relee** para producir el reporte final, en pantalla y en archivo.
- ✗ El reporte debe incluir reservas válidas e inválidas, el total facturado, y la reserva de mayor y menor importe.



Ejercicio 3: Puntos a repasar

- ✗ `fechaHora()` convierte fecha y hora en un long long comparable, para poder identificar extremos sin guardar texto.
- ✗ `esBisiesto()` y `diasDelMes()` validan que la fecha exista antes de aceptar la reserva como válida.
- ✗ `esLineaDeReserva()` distingue una línea de reserva de la línea de resumen mirando su primera clave.
- ✗ El archivo se recorre dos veces con propósitos distintos: primero para escribir el JSON, luego para leer y generar el reporte.

